

Unit-III: Number System, Base Conversion, BCD code, Excess-3 code, Gray Code, Review of Logic Gates, Concept of Universal Gates &, Boolean laws and theorems, SOP and POS representation of Boolean functions, Minimization of Boolean functions using K map, Basic Combinational Circuits: Half Adder, Full Adder, Subtractor, Sequential Circuits: Latch, Flip-Flops. Characteristic and Excitation Table of SR, JK, D and T Flip-flop. Concept of Master Slave Flip- Flop, Shift Registers. Give me proper extended notes of this without missing anything Give it to me in pdf form

<https://gemini.google.com/app/991f2173648ad2e>

User prompt: Unit-III: Number System, Base Conversion, BCD code, Excess-3 code, Gray Code, Review of Logic Gates, Concept of Universal Gates &, Boolean laws and theorems, SOP and POS representation of Boolean functions, Minimization of Boolean functions using K map, Basic Combinational Circuits: Half Adder, Full Adder, Subtractor, Sequential Circuits: Latch, Flip-Flops. Characteristic and Excitation Table of SR, JK, D and T Flip-flop. Concept of Master Slave Flip- Flop, Shift Registers. Give me proper extended notes of this without missing anything Give it to me in pdf form

Response: Digital Electronics (Unit-III)

1. Number Systems and Base Conversions

A positional number system represents numeric values using a set of symbols (digits) where the value of each symbol depends on its position relative to the radix point. Any number N with radix (base) r is represented as:

$$N = \sum_{i=-m}^{n-1} d_i \cdot r^i = d_{n-1}r^{n-1} + \dots + d_1r^1 + d_0r^0 + d_{-1}r^{-1} + \dots + d_{-m}r^{-m}$$

Where r is the base, d_i is the digit coefficient ($0 \leq d_i < r$), n is the number of integer digits, and m is the number of fractional digits.

Common Number Systems

Number System	Base (r)	Allowed Digits / Symbols	Example
Binary	2	0, 1	$(1011.01)_2$
Octal	8	0, 1, 2, 3, 4, 5, 6, 7	$(745.2)_8$
Decimal	10	0, 1, 2, 3, 4, 5, 6, 7, 8, 9	$(253.75)_{10}$
Hexadecimal	16	0–9, A(10), B(11), C(12), D(13), E(14), F(15)	$(2AF.4)_{16}$

Conversion Techniques

1. Any Base r to Decimal

Multiply each digit by its positional weight r^i and sum all terms.

Example: Convert $(1101.101)_2$ to Decimal:

$$\begin{aligned} N &= 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0 + 1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} \\ &= 8 + 4 + 0 + 1 + 0.5 + 0 + 0.125 = (13.625)_{10} \end{aligned}$$

2. Decimal to Any Base r

- **Integer Part:** Successive division by r . Collect remainders from bottom to top (Least Significant Bit/Digit at top, Most Significant Bit/Digit at bottom).
- **Fractional Part:** Successive multiplication by r . Collect integer overflows from top to bottom.

Example: Convert $(25.625)_{10}$ to Binary:

- **Integer (25):**

- $25 \div 2 = 12$, remainder 1 (LSB)
- $12 \div 2 = 6$, remainder 0
- $6 \div 2 = 3$, remainder 0
- $3 \div 2 = 1$, remainder 1
- $1 \div 2 = 0$, remainder 1 (MSB) $\rightarrow (11001)_2$
- *Fractional (0.625):*
 - $0.625 \times 2 = 1.25 \rightarrow$ integer 1
 - $0.25 \times 2 = 0.50 \rightarrow$ integer 0
 - $0.50 \times 2 = 1.00 \rightarrow$ integer 1 $\rightarrow (0.101)_2$
- *Result:* $(25.625)_{10} = (11001.101)_2$

3. Binary $\leftrightarrow$ Octal and Binary $\leftrightarrow$ Hexadecimal

- **Binary to Octal:** Group binary bits in sets of 3 starting from the radix point (left for integer, right for fractional, padding zeros as needed). Replace each 3-bit group with its octal equivalent ($2^3 = 8$).
- **Binary to Hexadecimal:** Group binary bits in sets of 4 starting from the radix point. Replace each 4-bit group with its hexadecimal equivalent ($2^4 = 16$).
- **Octal to Hexadecimal (and vice versa):** Convert through an intermediate binary representation.

2. Binary Codes

Binary Coded Decimal (BCD / 8421 Code)

- **Type:** Weighted code with weights 8, 4, 2, 1.
- **Principle:** Each decimal digit (0 through 9) is encoded as an independent 4-bit binary group.
- **Invalid States:** 4 bits can represent 16 states (0000 to 1111). States 1010 through 1111 (10 to 15 in decimal) are invalid in BCD.
- **BCD Addition Rule:** If the 4-bit binary sum yields an invalid code (> 9) or generates a carry out of the 4-bit group, add 6 (0110_2) to skip the six illegal states.

Excess-3 Code (XS-3)

- **Type:** Non-weighted, self-complementary code.
- **Derivation:** $XS-3 = BCD + 0011_2$ (+3 in decimal).
- **Self-Complementary Property:** The 9's complement of a decimal digit is directly obtained by inverting the bits (1's complement) of its Excess-3 code.
 - *Example:* Decimal 2 = $0010_{BCD} \rightarrow 0101_{XS-3}$.
 - 1's complement of 0101 = $1010_{XS-3} \rightarrow$ Decimal 7 ($9 - 2 = 7$).

Gray Code (Reflected Binary Code)

- **Type:** Non-weighted, unit-distance code.
- **Characteristic:** Successive code words differ by only a single bit. This eliminates spurious transitional states in electro-mechanical systems (e.g., optical shaft encoders).

Binary to Gray Conversion

Given an n -bit binary number $B = B_{n-1}B_{n-2} \dots B_1B_0$, the Gray code $G = G_{n-1}G_{n-2} \dots G_1G_0$ is generated by:

$$G_{n-1} = B_{n-1} \quad (\text{MSB remains identical})$$

$$G_i = B_{i+1} \oplus B_i \quad \text{for } i = n-2, \dots, 0$$

Gray to Binary Conversion

$$B_{n-1} = G_{n-1}$$

$$B_i = B_{i+1} \oplus G_i \quad \text{for } i = n - 2, \dots, 0$$

Decimal	4-Bit Binary	8421 BCD	Excess-3	Gray Code
0	0000	0000	0011	0000
1	0001	0001	0100	0001
2	0010	0010	0101	0011
3	0011	0011	0110	0010
4	0100	0100	0111	0110
5	0101	0101	1000	0111
6	0110	0110	1001	0101
7	0111	0111	1010	0100
8	1000	1000	1011	1100
9	1001	1001	1100	1101

3. Review of Logic Gates and Universal Gates

Basic and Derived Logic Gates

Gate	Logic Symbol Expression	Boolean Expression	Truth Table Summary
NOT	Inverter	$Y = \overline{A}$	Output is inversion of input
AND	Conjunction	$Y = A \cdot B$	$Y = 1$ if and only if all inputs are 1
OR	Disjunction	$Y = A + B$	$Y = 1$ if at least one input is 1
NAND	NOT-AND	$Y = \overline{A \cdot B}$	$Y = 0$ if and only if all inputs are 1
NOR	NOT-OR	$Y = \overline{A + B}$	$Y = 1$ if and only if all inputs are 0
XOR	Exclusive-OR	$Y = A \oplus B = \overline{A}B + A\overline{B}$	Output is 1 for odd number of 1s
XNOR	Equivalence	$Y = \overline{A \oplus B} = AB + \overline{A}\overline{B}$	Output is 1 when inputs are identical

Concept of Universal Gates

A **Universal Gate** can implement any arbitrary Boolean function without requiring any other gate type. Both **NAND** and **NOR** gates are functionally complete and serve as universal gates.

Gate Implementations Using Only NAND Gates

1. **NOT:** Tie inputs together: $Y = \overline{A \cdot A} = \overline{A}$.
2. **AND:** Invert output of a NAND: $Y = \overline{\overline{A \cdot B}} = A \cdot B$.
3. **OR:** Use De Morgan's theorem ($\overline{\overline{A} \cdot \overline{B}} = A + B$): Invert each input with a NAND, then pass to a NAND.
4. **NOR:** Follow an OR configuration with an inverter NAND.
5. **XOR:** Requires 4 NAND gates:

$$Y = A\overline{B} + \overline{A}B = \overline{\overline{A\overline{B}} \cdot \overline{\overline{A}B}}$$

6. **XNOR:** Requires 5 NAND gates (or 4 if inverted XOR is tapped).

Gate Implementations Using Only NOR Gates

1. **NOT:** Tie inputs together: $Y = \overline{A + A} = \overline{A}$.
2. **OR:** Invert output of a NOR: $Y = \overline{\overline{A + B}} = A + B$.
3. **AND:** Use De Morgan's theorem ($\overline{\overline{A} + \overline{B}} = A \cdot B$): Invert each input with a NOR, then pass to a NOR.

4. **NAND**: Follow an AND configuration with an inverter NOR.

5. **XOR**: Requires 5 NOR gates.

6. **XNOR**: Requires 4 NOR gates:

$$Y = AB + \overline{A}\overline{B} = \overline{\overline{A(\overline{A+B})} + \overline{B(\overline{A+B})}}$$

Minimum Number of Universal Gates Required

Target Gate	Equivalent NAND Gates	Equivalent NOR Gates
NOT	1	1
AND	2	3
OR	3	2
NAND	1	4
NOR	4	1
XOR	4	5
XNOR	5	4

4. Boolean Laws and Theorems

Boolean algebra operates on a binary set $B = \{0, 1\}$ with operators $+$ (OR) and $\cdot$ (AND).

Core Laws

Law Name	AND Form	OR Form
Identity Law	$A \cdot 1 = A$	$A + 0 = A$
Null / Dominance Law	$A \cdot 0 = 0$	$A + 1 = 1$
Idempotent Law	$A \cdot A = A$	$A + A = A$
Complement Law	$A \cdot \overline{A} = 0$	$A + \overline{A} = 1$
Involution (Double Negation)	$\overline{\overline{A}} = A$	—
Commutative Law	$A \cdot B = B \cdot A$	$A + B = B + A$
Associative Law	$(A \cdot B) \cdot C = A \cdot (B \cdot C)$	$(A + B) + C = A + (B + C)$
Distributive Law	$A + (B \cdot C) = (A + B)(A + C)$	$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
Absorption Law	$A \cdot (A + B) = A$	$A + (A \cdot B) = A$
Redundant Literal Rule	$A + \overline{A}B = A + B$	$A(\overline{A} + B) = AB$

De Morgan's Theorems

- **First Theorem**: The complement of a product of variables equals the sum of the complements:

$$\overline{A \cdot B \cdot C \dots} = \overline{A} + \overline{B} + \overline{C} + \dots$$

- **Second Theorem**: The complement of a sum of variables equals the product of the complements:

$$\overline{A + B + C + \dots} = \overline{A} \cdot \overline{B} \cdot \overline{C} \dots$$

Consensus Theorem

- **SOP Form**: $AB + \overline{A}C + BC = AB + \overline{A}C$ (The term BC is redundant).
- **POS Form**: $(A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C)$ (The factor $(B + C)$ is redundant).

5. SOP and POS Representations

Definitions: Minterms and Maxterms

- **Minterm (m_i):** A product (AND) of literals in which each variable appears exactly once in true or complemented form. A minterm evaluates to 1 for exactly one combination of input variables.
- **Maxterm (M_i):** A sum (OR) of literals in which each variable appears exactly once in true or complemented form. A maxterm evaluates to 0 for exactly one combination of input variables.
- **Relationship:** $m_i = \overline{M_i}$ and $M_i = \overline{m_i}$.

Canonical Forms

- **Canonical Sum of Products (Canonical SOP):** Formed by taking the logical OR of all minterms for which the output is 1:

$$F(A, B, C) = \sum m(1, 4, 5, 7)$$

- **Canonical Product of Sums (Canonical POS):** Formed by taking the logical AND of all maxterms for which the output is 0:

$$F(A, B, C) = \prod M(0, 2, 3, 6)$$

Standard vs. Canonical Forms

- A **Canonical Form** requires every term to contain all domain variables (e.g., $F = AB\overline{C} + \overline{A}BC$).
- A **Standard / Minimal Form** contains product or sum terms that may have one or more literals eliminated (e.g., $F = A\overline{C} + B$).

6. Minimization of Boolean Functions Using Karnaugh Maps (K-Maps)

A Karnaugh Map is a systematic graphical method for simplifying Boolean functions up to 6 variables. It arranges 2^n cells (where n is the number of variables) such that adjacent cells differ by only one bit position (**Gray code ordering**).

Adjacency and Gray Code Structure

Adjacent cells along rows or columns can be combined using the identity $X + \overline{X} = 1$ to eliminate redundant literals. Wrapping boundaries (top-to-bottom and leftmost-to-rightmost) are also contiguous.

2-Variable K-Map (A, B):

	B=0	B=1
A=0	m0	m1
A=1	m2	m3

3-Variable K-Map (A, BC):

	BC 00	01	11	10
A=0	m0	m1	m3	m2
A=1	m4	m5	m7	m6

4-Variable K-Map (AB, CD):

	CD 00	01	11	10
AB=00	m0	m1	m3	m2
AB=01	m4	m5	m7	m6
AB=11	m12	m13	m15	m14
AB=10	m8	m9	m11	m10

Grouping Rules

1. Group sizes must be powers of two (1, 2, 4, 8, 16).
2. Groups may wrap around edges and corners (e.g., cells m_0, m_2, m_8, m_{10} form a valid quad).
3. Each cell containing a 1 must be covered by at least one group.
4. Groups should be made as large as possible to eliminate the maximum number of variables.
5. **Prime Implicant (PI):** A rectangular group of cells that cannot be merged with any larger group.
6. **Essential Prime Implicant (EPI):** A prime implicant that includes at least one 1 that is not covered by any other prime implicant.

Don't Care Conditions (d or X)

Inputs that cannot occur or whose outputs do not impact system performance are designated as "Don't Care" states.

- In **SOP K-maps**, include d in groups only if it helps form a larger group of 1s. If it does not reduce expression size, treat it as 0.
- In **POS K-maps**, group d with 0s only if it helps form a larger product factor.

7. Basic Combinational Circuits

Combinational circuits are time-independent logic networks whose outputs depend solely on the present combination of input values, containing no internal feedback loops or memory elements.

Half Adder

A combinational circuit that adds two single-bit inputs A and B , generating a Sum (S) and a Carry (C).

Truth Table

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Boolean Equations

$$S = A \oplus B = \overline{A}B + A\overline{B}$$

$$C = A \cdot B$$

Full Adder

A combinational circuit that adds three single-bit inputs: two operands A and B , and a carry-in from a previous stage C_{in} .

Truth Table

A	B	C_{in}	Sum (S)	Carry Out (C_{out})
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

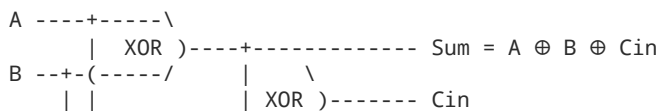
Boolean Equations

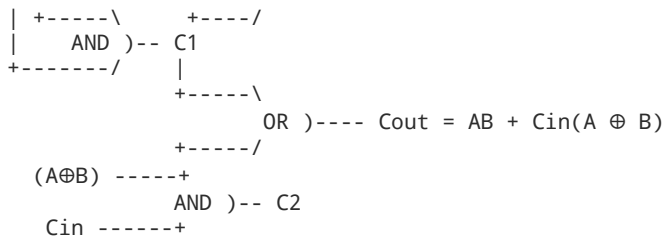
$$S = A \oplus B \oplus C_{in}$$

$$C_{out} = AB + BC_{in} + AC_{in} = AB + C_{in}(A \oplus B)$$

Realization Using Two Half Adders

1. **HA 1:** $S_1 = A \oplus B, C_1 = AB$.
2. **HA 2:** $S = S_1 \oplus C_{in} = A \oplus B \oplus C_{in}, C_2 = S_1 \cdot C_{in} = (A \oplus B)C_{in}$.
3. **Final Carry:** $C_{out} = C_1 + C_2 = AB + (A \oplus B)C_{in}$.





Half Subtractor

A combinational circuit that subtracts a single-bit subtrahend B from a minuend A , yielding Difference (D) and Borrow (B_{out}).

Truth Table

A	B	Difference (D)	Borrow (B_{out})
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

Boolean Equations

$$D = A \oplus B$$

$$B_{out} = \overline{A} \cdot B$$

Full Subtractor

A combinational circuit that performs subtraction involving three bits: minuend A , subtrahend B , and an incoming borrow B_{in} .

Truth Table

A	B	B_{in}	Difference (D)	Borrow Out (B_{out})
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

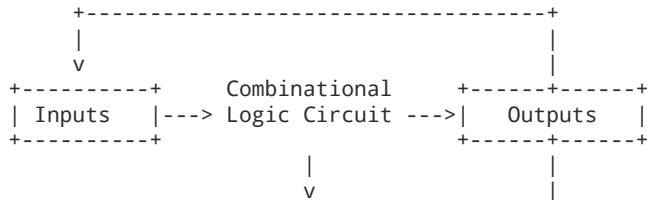
Boolean Equations

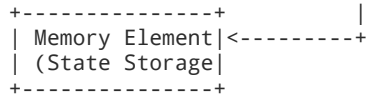
$$D = A \oplus B \oplus B_{in}$$

$$B_{out} = \overline{A}B + \overline{(A \oplus B)}B_{in} = \overline{A}B + \overline{A}B_{in} + BB_{in}$$

8. Sequential Circuits: Latches vs. Flip-Flops

Sequential circuits contain memory elements. Their output depends on both current inputs and the previous state stored within the system.





Comparison: Latch vs. Flip-Flop

Parameter	Latch	Flip-Flop
Triggering Type	Level-triggered (Transparent during active pulse)	Edge-triggered (Transitions on rising/falling edge)
Clock Signal	Typically uses an enable signal; no clock needed	Uses a periodic synchronizing clock signal
Circuit Design	Simpler, uses fewer logic gates	More complex (built using cross-coupled latches)
Susceptibility	Vulnerable to race-around conditions and noise	Immune to level glitches; transitions only at clock edges
Primary Use	High-speed cache buffers, asynchronous pipelines	Synchronous counters, shift registers, finite state machines

SR Latch Operations

An SR Latch can be built using cross-coupled NOR or NAND gates.

Active-High NOR SR Latch

- $S = 1, R = 0 \implies Q = 1, \overline{Q} = 0$ (Set)
- $S = 0, R = 1 \implies Q = 0, \overline{Q} = 1$ (Reset)
- $S = 0, R = 0 \implies Q_{n+1} = Q_n$ (No change / Storage)
- $S = 1, R = 1 \implies Q = 0, \overline{Q} = 0$ (Invalid / Forbidden state, violates $Q = \overline{\overline{Q}}$)

9. Flip-Flop Types: Tables and Equations

1. SR Flip-Flop (Set-Reset)

Addresses latch transparency by qualifying inputs with a clock edge.

- Logic Function Table:

CLK	S	R	Q_{n+1}	Description
↑	0	0	Q_n	Hold / No Change
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	Undefined	Invalid (Forbidden)

- Characteristic Table:

Q_n	S	R	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	X
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	X

- Characteristic Equation:

$$Q_{n+1} = S + \overline{R}Q_n \quad (\text{Constraint: } S \cdot R = 0)$$

- Excitation Table:

Q_n	Q_{n+1}	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

2. JK Flip-Flop

Resolves the invalid state of the SR flip-flop by feeding back complementary outputs to the input gates. When $J = K = 1$, the device toggles.

- Logic Function Table:

CLK	J	K	Q_{n+1}	Description
↑	0	0	Q_n	Hold
↑	0	1	0	Reset
↑	1	0	1	Set
↑	1	1	$\overline{Q}_n$	Toggle

- Characteristic Table:

Q_n	J	K	Q_{n+1}
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

- Characteristic Equation:

$$Q_{n+1} = J\overline{Q}_n + \overline{K}Q_n$$

- Excitation Table:

Q_n	Q_{n+1}	J	K
0	0	0	X
0	1	1	X
1	0	X	1
1	1	X	0

3. D Flip-Flop (Data / Delay)

Eliminates intermediate ambiguous states by setting input $R = \overline{S}$. Used in pipeline registers, memory buffers, and counters.

- Logic Function Table:

CLK	D	Q_{n+1}	Description
↑	0	0	Reset
↑	1	1	Set

- Characteristic Table:

Q_n	D	Q_{n+1}
0	0	0
0	1	1
1	0	0
1	1	1

- Characteristic Equation:

$$Q_{n+1} = D$$

- Excitation Table:

Q_n	Q_{n+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

4. T Flip-Flop (Toggle)

Created by tying the inputs of a JK flip-flop together ($J = K = T$). If $T = 1$, the state inverts on each active clock edge. Useful in frequency division and binary ripple counters.

- Logic Function Table:

CLK	T	Q_{n+1}	Description
↑	0	Q_n	Hold
↑	1	$\overline{Q}_n$	Toggle

- Characteristic Table:

Q_n	T	Q_{n+1}
0	0	0
0	1	1
1	0	1
1	1	0

- Characteristic Equation:

$$Q_{n+1} = T \oplus Q_n = T\overline{Q}_n + \overline{T}Q_n$$

- Excitation Table:

Q_n	Q_{n+1}	T
0	0	0
0	1	1

Q_n	Q_{n+1}	T
1	0	1
1	1	0

10. Master-Slave Flip-Flop & Race-Around Condition

The Race-Around Condition in Level-Triggered JK Flip-Flops

In a level-triggered JK flip-flop, when $J = 1$ and $K = 1$, the output toggles every propagation delay time (t_{pd}).

If the clock pulse width t_w is significantly longer than the propagation delay of the gates ($t_w > t_{pd}$):

1. The outputs toggle from $0 \rightarrow 1$.
2. This updated output feeds back to the inputs while the clock is still high.
3. The outputs toggle again ($1 \rightarrow 0$), repeating in an uncontrolled cycle throughout the clock pulse.
4. At the falling edge of the clock, the final output state is indeterminate. This unstable behavior is known as the **Race-Around Condition**.

$$\text{Condition for Race-Around: } t_{pd} \leq t_w \leq T/2$$

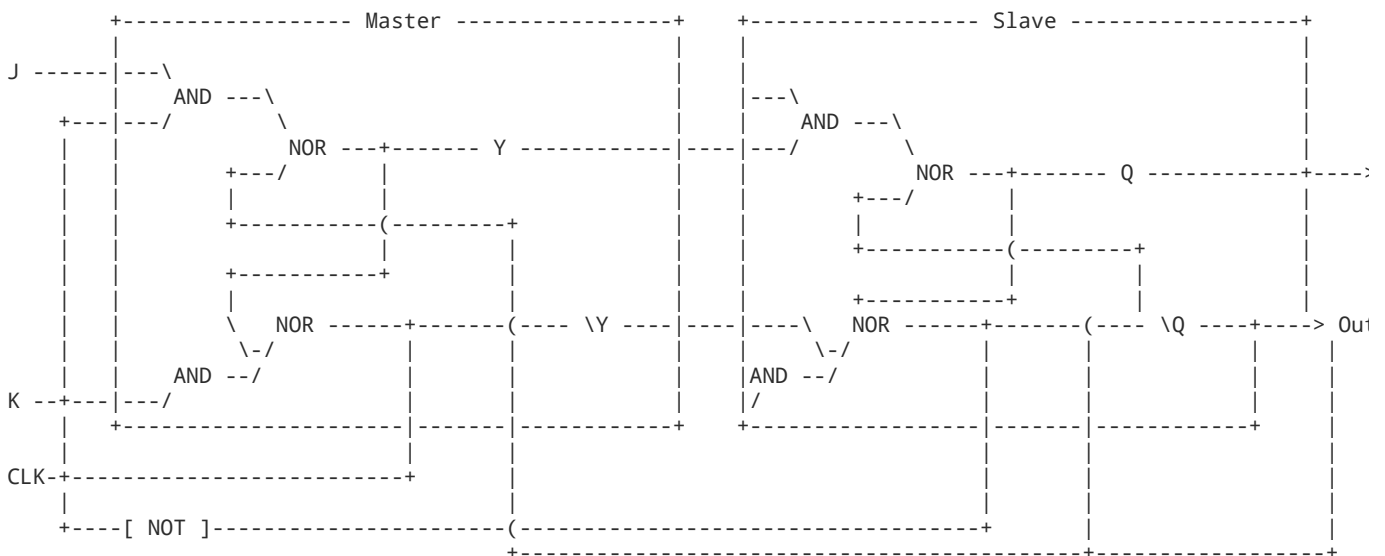
Remediation Methods

- Use edge-triggering circuits (differentiating networks).
- Enforce propagation delay condition $t_{pd} > t_w$.
- Use a **Master-Slave JK Flip-Flop** configuration.

Master-Slave JK Flip-Flop Architecture

The Master-Slave circuit combines two distinct latches configured in cascade with complementary clocks:

1. **Master Stage:** A clocked JK latch triggered on the positive clock level ($\text{CLK} = 1$).
2. **Slave Stage:** An SR latch triggered on the inverted clock level ($\overline{\text{CLK}} = 1$, or $\text{CLK} = 0$).



Operational Sequence

1. **When $\text{CLK} = 1$:**
 - The Master is active and responds to inputs J and K .
 - The Master updates internal intermediate outputs Y and $\overline{Y}$.
 - The inverter drives the Slave clock to 0, isolating the Slave. Outputs Q and $\overline{Q}$ remain unchanged.
2. **When CLK transitions from 1 $\rightarrow$ 0:**

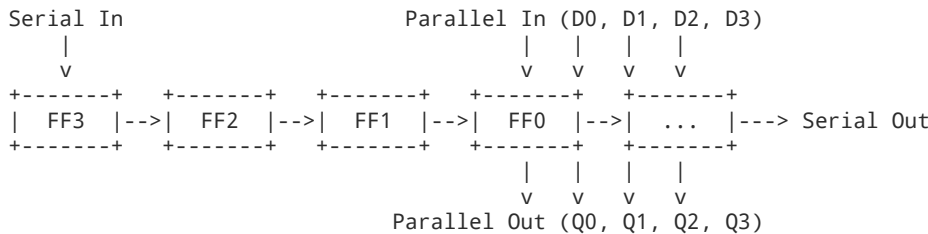
- The Master inputs are disabled, locking in states Y and $\overline{Y}$.
- The Slave clock becomes 1. The Slave latch reads Y and $\overline{Y}$, transferring these states to final outputs Q and $\overline{Q}$.

3. **Result:** Because data transfers to the output only during the clock transition (when the Master is locked), feedback cannot alter the inputs during the same cycle. This completely eliminates the race-around condition.

11. Shift Registers

A register is an array of flip-flops used to store multi-bit binary data. A **Shift Register** can also move stored bits laterally one position at a time during each clock pulse. An n -bit shift register requires n flip-flops.

Operational Classifications



1. Serial-In Serial-Out (SISO)

- **Data Entry:** One bit per clock cycle via a single input line.
- **Data Extraction:** One bit per clock cycle from the terminal flip-flop output.
- **Performance:** Requires n clock cycles to load an n -bit word, and $n - 1$ additional cycles to read it out serially ($2n - 1$ clock cycles total). Provides the slowest transfer rate, but uses the fewest transmission lines.

2. Serial-In Parallel-Out (SIPO)

- **Data Entry:** Serial stream shifted in bit-by-bit.
- **Data Extraction:** All outputs ($Q_0, Q_1, \dots, Q_{n-1}$) are tapped simultaneously.
- **Performance:** Requires n clock cycles to fully load n bits. Once loaded, the parallel word is available immediately. Frequently used in serial communication receivers (deserializers).

3. Parallel-In Serial-Out (PISO)

- **Data Entry:** All stages are loaded simultaneously using combinational steering logic qualified by a $\text{Shift}/\overline{\text{Load}}$ control line.
- **Data Extraction:** Read serially via the final flip-flop's output.
- **Performance:** Requires 1 clock cycle to load in parallel, and $n - 1$ clock cycles to shift the data out serially. Used in serial communication transmitters (serializers).

4. Parallel-In Parallel-Out (PIPO)

- **Data Entry:** All bits are loaded in parallel on a single clock pulse.
- **Data Extraction:** All outputs are read in parallel immediately.
- **Performance:** Requires only 1 clock cycle to load and read out (0 latency once clocked). Acts primarily as a multi-bit memory storage buffer.

Universal Shift Register

A Universal Shift Register can perform all register operations based on external control inputs: hold state, shift right, shift left, and load parallel data.

A 4-bit Universal Shift Register uses four D flip-flops and four 4-to-1 multiplexers with select lines S_1, S_0 :

Multiplexer Inputs (4:1 MUX):
 I0: Retain current state (Connects to Q of corresponding FF)
 I1: Shift-Right operation (Connects to Q of FF to the left, or Serial Input)
 I2: Shift-Left operation (Connects to Q of FF to the right, or Serial Input)
 I3: Parallel Load (Connects to external parallel input line D)

S_1	S_0	Operational Mode	Description
0	0	Hold (No Change)	Output recirculates back into its own flip-flop stage
0	1	Shift Right	Bits move down toward the LSB ($Q_{i+1} \rightarrow Q_i$)
1	0	Shift Left	Bits move up toward the MSB ($Q_i \rightarrow Q_{i+1}$)
1	1	Parallel Load	Parallel inputs D_3, D_2, D_1, D_0 load directly into the register

Tip for saving as PDF: Press Ctrl + P (or Cmd + P on macOS) in your browser or Markdown reader, choose **Save as PDF**, and adjust options to ensure all tables and code diagrams render cleanly.
